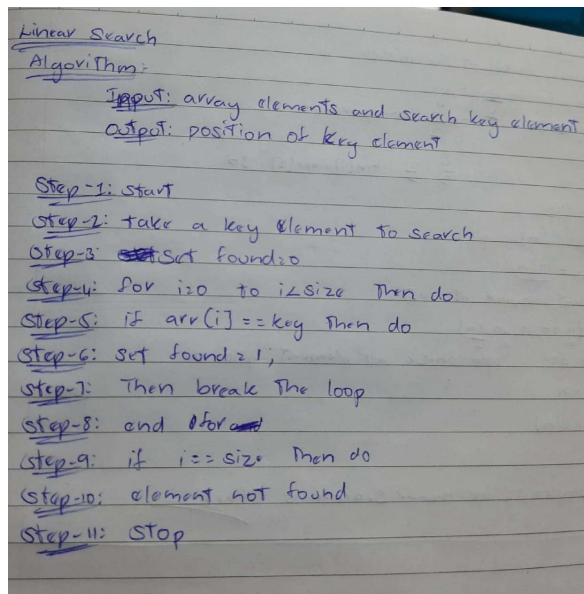


PRACTICAL-02

AIM: Implementation and Time analysis of linear and binary search algorithm.

1.) LINEAR SEARCH

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

int main() {

    int arr[20], n, key, i;

    cout << "Enter the number of elements: ";

    cin >> n;

    cout << "Enter the array elements: ";

    for (int i = 0; i < n; i++) {

        cin >> arr[i];

    }

    cout << "Enter the element to search: ";

    cin >> key;
```

```

for (int i = 0; i < n; i++) {
    if (arr[i] == key) {
        cout << "Element found at index " << i;
        return 0;
    }
}

cout << "Element not found";

return 1;
}

```

OUTPUT:

```

Output

Enter the number of elements: 5
Enter the array elements: 9
1
6
7
2
Enter the element to search: 2
Element found at index 4

=== Code Execution Successful ===

```

TIME COMPLEXITY:

Time complexity

```

for (i=0; i<n; i++){      → n
    if (arr[i] == key)    → n
        cout << "element found" → 1
        found = 1;        → 1
        break;            → 1

```

$$T(n) = n + n + 1 + 1 + 1$$

$$= 2n + 3$$

$$= 2n$$

$$T(n) = O(n)$$

1) Best case - $O(1)$

ex: arr = {10, 20, 30, 40, 50}

key = 10, There is only one element compare

$$T(n) = 1$$

Time complexity = $O(1)$

2) Average case - $O(n)$

element found at in middle

ex: arr = {10, 20, 30, 40, 50}

key = 30, $\frac{n}{2} \rightarrow \frac{5}{2}$ compare

$\frac{5}{2} = 2$ arr[2] = 30

$$T(n) = n/2$$

$$T(n) = O(n)$$

3) worst case - $O(n)$

element found at last

ex: arr = {10, 20, 30, 40, 50}

key = 50, we check all element upto n

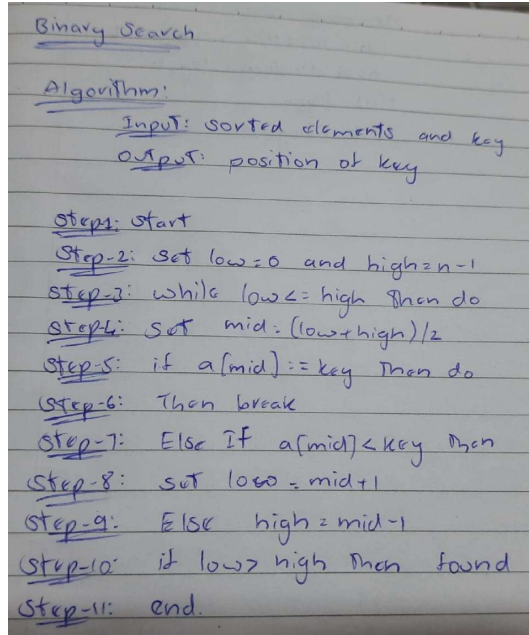
$$T(n) = n$$

$$T(n) = O(n)$$

Time complexity for linear search is $O(n)$

2.) BINARY SEARCH

ALGORITHM:



PROGRAM:

```
#include <iostream>

using namespace std;

int main()
{
    int n, key;
    int low, high, mid;

    cout << "Enter the number of elements: ";
    cin >> n;

    int a[n];

    cout << "Enter the sorted array elements: ";
    for (int i = 0; i < n; i++)
    {
        cin >> a[i];
    }

    cout << "Enter the key element: ";
    cin >> key;
```

```

low = 0;

high = n - 1;

while (low <= high)
{
    mid = (low + high) / 2;

    if (a[mid] == key)
    {
        cout << "Element found at position " << mid << endl;

        break;
    }

    else if (a[mid] < key)
    {
        low = mid + 1;
    }

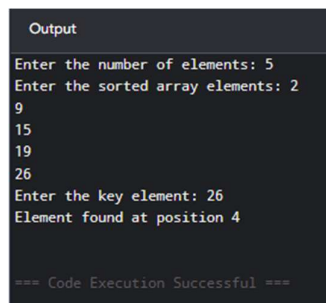
    else
    {
        high = mid - 1;
    }
}

if (low > high)
{
    cout << "Element not found" << endl;
}

return 0;
}

```

OUTPUT:



```

Output
Enter the number of elements: 5
Enter the sorted array elements: 2
9
15
19
26
Enter the key element: 26
Element found at position 4

=== Code Execution Successful ===

```

TIME COMPLEXITY:

Time complexity-

```
low = 0; high = n-1; → 1
while (low ≤ high) { → log n
    mid = (low+high)/2; → log n
    if (a[mid] == key) → log n
        break; → 1
    else if (a[mid] < key) → log n
        low = mid+1; → log n
    else
        high = mid-1; → log n
}
```

$T(n) = 1 + \log n + \log n + \log n + 1 + 1 + \log n + \log n + \log n$
 $= 2 + 6 \log n$
 $T(n) = O(\log n)$ because in every iteration, array divides into half

i) Best case - $O(1)$ ii) Average case - $O(\log n)$

$mid = (low+high)/2 \rightarrow 1$ $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \dots 1$
 $if (a[mid] == key) \rightarrow 1$ no. of divisions = $\log_2 n$
 $T(n) = 1 + 1 = 2$ $T(n) = \log_2 n$
 $T(n) = O(1)$ $T(n) = O(\log n)$

2) worst case - $O(\log n)$

$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \dots 1$
 $T(n) = \log_2 n$
 $T(n) = O(\log n)$